

Problem J1. Roller Coaster Ride

The ride can hold C cars with P people per car, so its capacity is $C \times P$. If $N \leq C \times P$, print **yes**; otherwise print **no**.

Complexity. This is an $O(1)$ calculation.

Problem J2. Donut Shop

Keep one variable for the current number of donuts. Read the initial number D , then process the E events. Each event consists of a sign and an integer amount. If the sign is $+$, add the amount; if it is $-$, subtract it. Print the final total.

The sign should be read as a character or string, while the amount should be read as an integer.

Complexity. The running time is $O(E)$, and the memory usage is $O(1)$.

Problem J3. Product Codes

For each original product code, scan the characters from left to right. Uppercase letters are kept in order and appended to the output prefix. Lowercase letters are ignored. Integers, including negative and multi-digit integers, are accumulated into a single sum that is printed after the uppercase prefix.

The main difficulty is tokenization. When a digit or minus sign begins an integer token, continue reading digits until the token ends, convert the token to an integer, and add it to the running sum. A state-machine approach works well: the current state records whether we are reading letters, a positive number, or a negative number.

After the scan finishes, remember to add the last pending number if the code ended while reading digits.

Complexity. Each character is processed once, so the running time is linear in the total length of all codes.

Problem J4. Sunny Days

Exactly one day in the data is wrong, so one character can be flipped between **S** and **P**. A brute-force solution tries every possible flip and recomputes the longest consecutive block of **S**; this is too slow when N is large.

For the full solution, process the string as blocks of consecutive identical characters. Apart from the all-sunny special case, the best answer comes from either extending one block of **S** by changing an adjacent **P**, or joining two **S** blocks separated by a single **P**. Therefore it is enough to examine at most three consecutive blocks at a time.

If the string is all **S**, flipping one day to **P** means the answer is $N - 1$.

Complexity. The block scan runs in $O(N)$ time and uses $O(N)$ or $O(1)$ memory depending on whether block lengths are stored.

Problem J5. Connecting Territories

Let $dp[r][c]$ be the minimum cost of a path that reaches cell (r, c) . To reach a cell in row r , Ava must come from one of at most three cells in row $r - 1$: the same column, one column to the left, or one column to the right. Thus,

$$dp[r][c] = G[r][c] + \min(dp[r-1][c-1], dp[r-1][c], dp[r-1][c+1]),$$

ignoring out-of-bounds terms.

This recurrence can be implemented recursively with memoization, or iteratively row by row. The answer is the minimum value in the bottom row.

For the largest grids, storing the whole grid or all DP rows is unnecessary. Only the previous row of DP values and the current row are needed.

Complexity. The iterative DP uses $O(RC)$ time and $O(C)$ memory.